Day 101 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 15, Day 4 reference

Watch: Text Summarization Explained: Abstractive vs Extractive (SystemDR) | Abstractive Text Summarization using Transformers-BART Model (ProjectPro)

`day101_summarization.py`, segment by segment, in the order `main()` uses things. Day 97's ROUGE and Day 99's bootstrap are imported unchanged; T5 comes from the library; the rest is from scratch. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: d = PageRank damping (0.85) · W = sentence-to-sentence vote matrix · r = sentence scores · idf = inverse document frequency

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
sys.path.insert(0, os.path.join(HERE, "..", "week-14-mini-gpt-language-modeling"))
with contextlib.redirect_stdout(io.StringIO()):
    import day97_lm_evaluation as d97
    import day99_exercise as d99
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")
hf_logging.set_verbosity_error(); hf_logging.disable_progress_bar()
hub_logging.set_verbosity_error(); disable_progress_bars()
torch.set_num_threads(4)
MODEL = "google-t5/t5-small"
N_ARTICLES, K = 100, 3
SYSTEMS = ["lead-3", "TextRank", "T5-small", "oracle"]
```

- `sys.path.insert(0, os.path.join(HERE, "..", "week-14-mini-gpt-language-modeling"))` — a path into another week's folder, so Day 97 can be imported.
- `sys.stdout.reconfigure(encoding="utf-8")` — the articles contain curly quotes; Windows would otherwise write them in a legacy encoding.

Segment 1 — Sentences

split_sentences: rules, not a model

CODE

```
ABBREV = {"mr", "mrs", "ms", "dr", "prof", "st", "jr", "sr", "gen", "gov", "sen", "rep", "lt",
    "col", "sgt", "capt", "u.s",
          "u.k", "no", "vs", "inc", "co", "corp", "ltd", "jan", "feb", "mar", "apr", "aug",
              "sept", "oct", "nov", "dec"}
def split_sentences(text):
    parts = re.split(r"(?<=[.!?])\s+(?=[\"'‘“(]?[A-Z0-9])", text.strip())
    out = []
    for p in parts:
        prev = out[-1].split()[-1].lower().rstrip(".") if out else ""
        if out and (prev in ABBREV or re.fullmatch(r"[a-z]", prev)):
            out[-1] += " " + p
        else:
            out.append(p)
    return [s.strip() for s in out if len(s.split()) >= 3]
```

- `re.split(r"(?<=[.!?])\s+(?=...[A-Z0-9])", ...)` — split at spaces that follow a stop (lookbehind) and precede a capital or digit (lookahead).
- `prev in ABBREV or re.fullmatch(r"[a-z]", prev)` — 'Mr.' or a one-letter initial is not a sentence end: join the pieces back.
- `len(s.split()) >= 3` — fragments of one or two words are dropped.

Segment 2 — The systems

tfidf and textrank

MATH

$$r \leftarrow (1-d)/n + d\, W^{T} r, \quad W_{ij} = S_{ij} / \Sigma_j S_{ij}, \quad S = \text{cosine of TF-IDF vectors}$$

CODE

```
def tfidf(sents):
    toks = [d97.words(s) for s in sents]
    vocab = {w: i for i, w in enumerate(sorted({w for t in toks for w in t}))}
    tf = np.zeros((len(sents), len(vocab)))
    for r, t in enumerate(toks):
        for w in t:
            tf[r, vocab[w]] += 1
    idf = np.log(len(sents) / (1 + (tf > 0).sum(0))) + 1
    v = tf * idf
    return v / (np.linalg.norm(v, axis=1, keepdims=True) + 1e-12)
def textrank(sents, k=K, d=0.85, iters=100):
    V = tfidf(sents); S = V @ V.T
    np.fill_diagonal(S, 0)
    W = S / (S.sum(1, keepdims=True) + 1e-12)
    r = np.ones(len(sents)) / len(sents)
    for _ in range(iters):
        r = (1 - d) / len(sents) + d * W.T @ r
    top = sorted(np.argsort(-r)[:k])
    return " ".join(sents[i] for i in top), r
```

- `np.log(len(sents) / (1 + (tf > 0).sum(0))) + 1` — idf: words in few sentences weigh more; +1 keeps every weight positive.
- `V = tfidf(sents); S = V @ V.T` — unit-length rows, so the matrix product is every pair's cosine similarity.

- `r = (1 - d) / len(sents) + d * W.T @ r` — one step of PageRank's power iteration; 100 steps converge.
- `sorted(np.argsort(-r)[:k])` — the k highest-scoring sentences, put back in article order.

oracle: the best three sentences, by looking at the answer

CODE

```
def oracle(sents, ref, k=K):
    chosen, best = [], 0.0
    for _ in range(k):
        gains = [(d97.rouge_n(" ".join(sents[j] for j in sorted(chosen + [i])), ref, 1) +
                  d97.rouge_n(" ".join(sents[j] for j in sorted(chosen + [i])), ref, 2), i) for i
                      in range(len(sents)) if i not in chosen]
        if not gains or max(gains)[0] <= best:
            break
        best, i = max(gains); chosen.append(i)
    return " ".join(sents[j] for j in sorted(chosen))
```

- `max(gains)` — tuples compare by their first element: the largest ROUGE gain, then its index.
- `if not gains or max(gains)[0] <= best: break` — stop adding sentences once none improves the score.

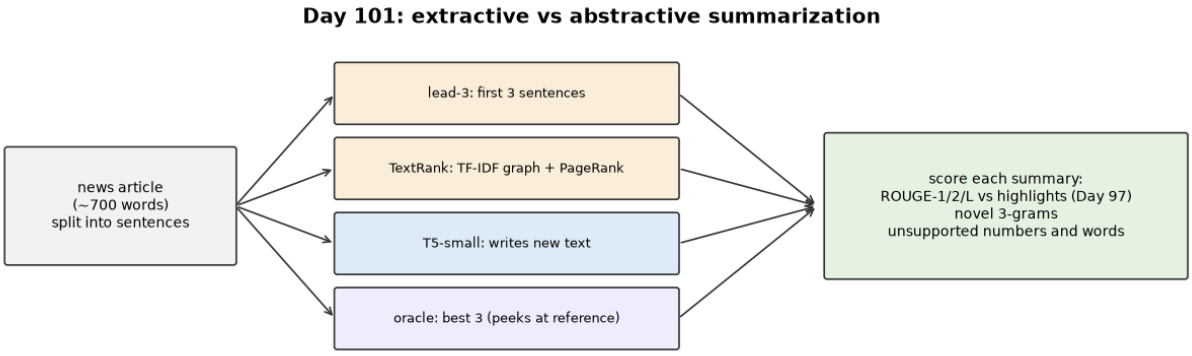
t5_summaries: generation with the model's own settings

CODE

```
def t5_summaries(articles):
    tok = AutoTokenizer.from_pretrained(MODEL); model =
        AutoModelForSeq2SeqLM.from_pretrained(MODEL).eval()
    cfg = model.config.task_specific_params["summarization"]
    out, t0 = [], time.perf_counter()
    with torch.no_grad():
        for a in articles:
            enc = tok(cfg["prefix"] + a, truncation=True, max_length=512, return_tensors="pt")
            ids = model.generate(**enc, num_beams=cfg["num_beams"], max_length=cfg["max_length"],
                min_length=cfg["min_length"],
                                 length_penalty=cfg["length_penalty"],
                                     no_repeat_ngram_size=cfg["no_repeat_ngram_size"],
                                 early_stopping=cfg["early_stopping"])
            out.append(tok.decode(ids[0], skip_special_tokens=True))
    return out, cfg, (time.perf_counter() - t0) / len(articles), model
```

- `AutoModelForSeq2SeqLM` — an encoder-decoder model: it reads the article and writes a new text token by token.
- `model.config.task_specific_params["summarization"]` — the prefix and decoding settings T5's authors saved with the model.
- `model.generate(..., num_beams=..., no_repeat_ngram_size=...)` — beam search, never repeating any 3-gram in the output.
- `tok.decode(ids[0], skip_special_tokens=True)` — token ids back to text, without the padding and end tokens.

SYSTEM DIAGRAM



Four ways to summarise one article.

Segment 3 — Checks ROUGE cannot do

novel_trigrams and unsupported

CODE

```
def novel_trigrams(summary, article):
    s, a = d97.words(summary), set(d97.ngrams(d97.words(article), 3))
    tri = [tuple(s[i:i + 3]) for i in range(len(s) - 2)]
    return np.mean([t not in a for t in tri]) if tri else 0.0
def unsupported(summary, article):
    nums = set(re.findall(r"\d[\d,.]*\d|\d", summary)) - set(re.findall(r"\d[\d,.]*\d|\d",
        article))
    stems = {w[:5] for w in re.findall(r"[a-z]+", article.lower())}
    new = {w for w in re.findall(r"[a-z]+", summary.lower()) if len(w) >= 4 and w[:5] not in
        stems}
    return nums | new
```

- `set(d97.ngrams(d97.words(article), 3))` — Day 97's n-gram counter, turned into a set of the article's word triples.
- `re.findall(r"\d[\d,.]*\d|\d", ...)` — numbers such as 7, 1,200 and 3.5.
- `{w[:5] for w in ...}` — a set comprehension of five-letter stems: 'suffered' and 'suffer' share 'suffe'.
- `nums | new` — the union of two sets.

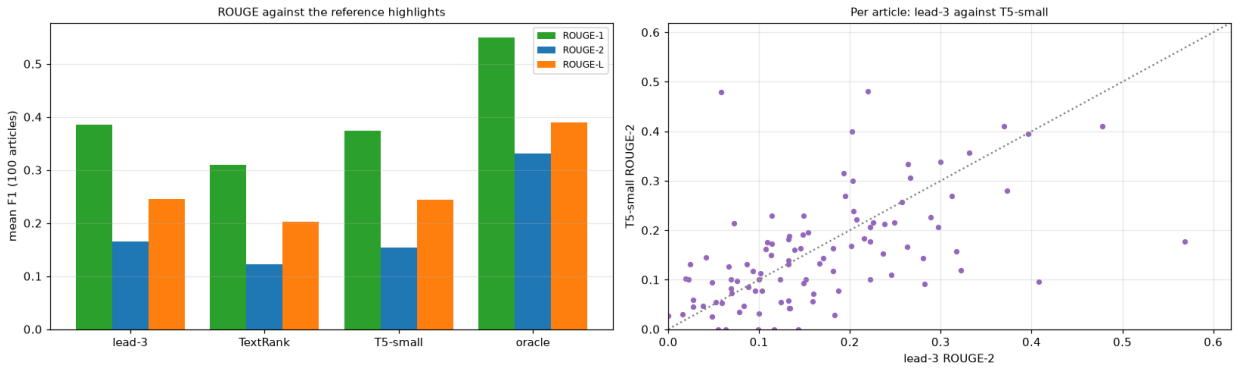
demo_rouge: per-article scores, intervals and pairs

CODE

```
scores = {}
print(f"\n{'system':<10}{'ROUGE-1':>9}{'ROUGE-2':>9}{'ROUGE-L':>9}{'ROUGE-2 95%
    interval':>24}{'words':>7}")
for name in SYSTEMS:
    sc = np.array([(d97.rouge_n(h, r, 1), d97.rouge_n(h, r, 2), d97.rouge_l(h, r)) for h, r in
        zip(out[name], refs)])
    scores[name] = sc; lo, hi = d99.bootstrap_ci(sc[:, 1])
    print(f"{name:<10}{sc[:, 0].mean():>9.3f}{sc[:, 1].mean():>9.3f}{sc[:, 2].mean():>9.3f}
        [{lo:.3f}, {hi:.3f}]"
          f"{np.mean([len(h.split()) for h in out[name]]):>7.0f}")
print("\npaired differences in ROUGE-2 (Day 99's paired bootstrap): mean, 95% interval, share of
    resamples ahead")
for a, b in (("lead-3", "TextRank"), ("lead-3", "T5-small"), ("T5-small", "TextRank"), ("oracle",
    "lead-3")):
    m, lo, hi, share = d99.paired_bootstrap(scores[a][:, 1], scores[b][:, 1])
```

- `np.array([(d97.rouge_n(h, r, 1), ...) for h, r in zip(...)])` — an (articles, 3) array: ROUGE-1, -2, -L for every article.
- `d99.bootstrap_ci(sc[:, 1])` — Day 99's part: the interval for the mean ROUGE-2.
- `d99.paired_bootstrap(a, b)` — Day 99's part: two systems compared on the same articles.

REAL GRAPH



Real graph from the script: ROUGE by system, and lead-3 against T5-small article by article.

Quick Syntax Glossary — new or reinforced constructs

- `(?<=...)` and `(?=...)` — regex lookbehind and lookahead: conditions that match no characters.
- `np.fill_diagonal(S, 0)` — set the diagonal of a square matrix in place.
- `model.generate(...)` — the library's decoding loop (greedy, sampling or beam search).
- `tok.decode(ids, skip_special_tokens=True)` — ids back to text.
- `set_a | set_b` — union of two sets.
- `sys.stdout.reconfigure(encoding="utf-8")` — change how printed text is encoded.

End of Day 101 syntax reference. For the experiments and what they showed, see the Day 101 complete guide.